Qawafel technical assessment

Financing Evidence Workbench

Abdulrahman Alrehaili | Graduate Software Engineer | Version 1.2 | 19 October 2026

Task 1 - Requirements

Objective and scope

Prepare reviewable financing evidence: an accountant supplies files, a Qawafel reviewer resolves discrepancies, and the bank analyst decides. MVP: one fictional SAR entity/account, sales invoices, no opening receivables/credit notes, Oct 2025-Sep 2026. Real provider consent and actual bank submission are production design; mock-source read consent/scheduling is implemented.

User stories and acceptance criteria

User / story	Acceptance criteria
Accountant: upload evidence	XLSX/CSV + supported PDF return ID and hashes. Invalid schema/format, excessive size or incomplete bank periods return actionable errors.
Accountant: fix problems	Invalid rows show original row/reason. Duplicates count once; duplicates/missing months block readiness. Corrected files produce a new analysis.
Reviewer: verify collections	Match reference, timing and available gross balance; support partial receipts. Exclude owner capital. Unallocated receipts block readiness; monthly bank balances reconcile/carry forward.
Analyst: inspect evidence	Show 90+ overdue, customer arrears, collection coverage and not-overdue balances. Model flags retain source rows. No approval/default probability.
Reviewer: reproduce findings	Export hashes, cutoff, version, ledger and issues. Identical inputs/version reuse ID and calculations. Any blocker means Needs data correction.
Reviewer: approve handoff (design)	Approve immutable version with named reviewer/consent. Changes invalidate approval. One bank idempotency key; ambiguous results remain unverified.

Questions before building

1. Which financing product, amount, term and bank eligibility/reporting rules apply?

2. Which ERPs, accounts, invoice types, currencies and PDF layouts occur? How frequent are partial/batch/advance payments or missing references?

3. How are reporting periods, genuine zero-sales months and opening receivables established?

4. Who may upload/approve/consent, and what access, retention and data-sharing requirements apply?

5. What are preparation/review volumes and times? Which bank source verifies status, and who owns escalation?

Data risks and responses

- **Duplicates:** count repeated IDs once and block; investigate conflicts and cross-ID similarities.
- **Gaps/entity/currency:** block missing months; SAR-only MVP; verify ownership/zero-sales in production.
- **Bad fields:** quarantine dates, amounts, formulas and VAT errors; normalize tiny float noise only.
- **PDFs:** reject unsupported scans/layouts or balance mismatch; add reviewed bank/OCR adapters.
- **Allocation:** exclude owner capital; block unmatched receipts; refunds/opening receivables need new ledgers.
- **Misuse/security:** no lending decision; production adds scoped access, scanning, encryption and audit.

Task 2 - Technical specification

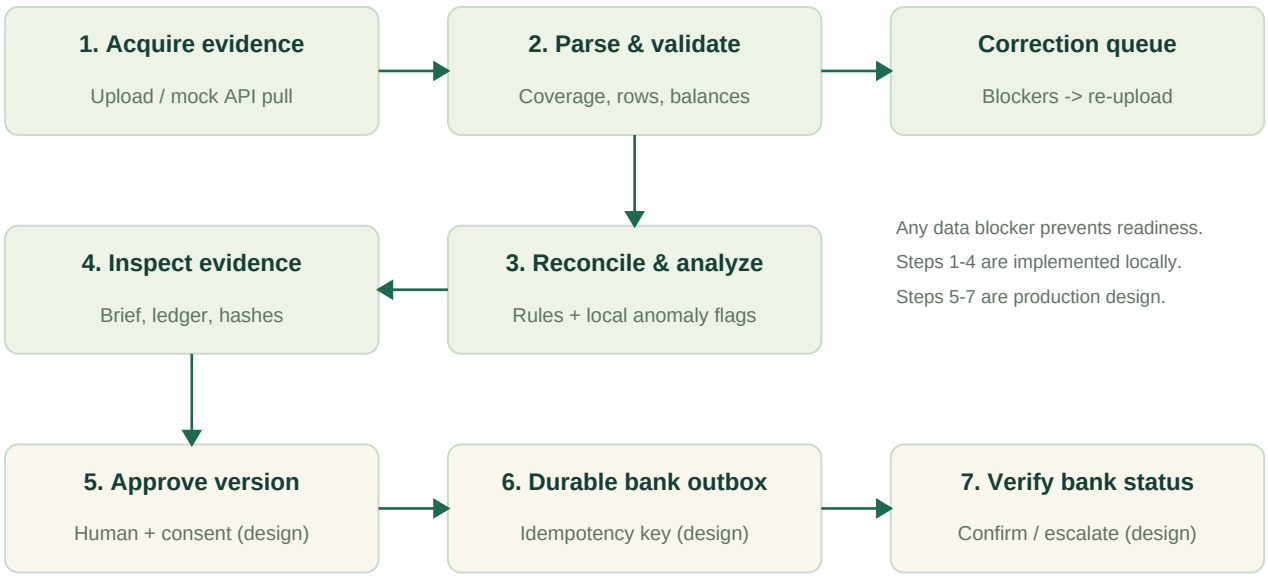
Architecture and data model

Implemented: FastAPI, openpyxl, pdfplumber, scikit-learn, plain HTML/CSS/JS and SQLite immutable JSON snapshots. Uploaded originals are parsed in memory and discarded; results and hashes stay in local SQLite. No runtime LLM, model key or real bank credentials. Mock source HTTP APIs and a persisted scheduler are implemented. Production would add PostgreSQL, encrypted document storage, a job queue and scoped authorization. Full schema: docs/schema.sql.

Entity / key	Fields and constraints
Organization / FinancingRequest	org_id; legal identity; request_id, reporting window, requested halalas, currency, state. Tenant owns requests; dates valid and requested amount positive.
Document	request_id, version, type, SHA-256, object key, parser version. Immutable sources; byte hashes prove identity, not authenticity.
PackageVersion	request_id + unique version, state, calculation version, evidence JSON, timestamp. Changed evidence invalidates approval.
Invoice	package_id + unique external ID, customer, issue/due, net/VAT halalas, document/row. Positive sales; due >= issue.
BankTransaction / Allocation	Unique package transaction ID, date, signed halalas, type, reference, source. Allocation joins invoice and receipt; transactional locking enforces same package and sum limits.
Issue / Review	Package, severity, source locator, resolution; reviewer, decision, consent reference, timestamp. Unresolved blockers prevent approval.
Submission / Observation / Outbox	Unique package and idempotency key, bank reference; append-only status/time/source; retry count and dispatch time. A transport timeout is an unknown outcome.
AuditEvent	Org, actor, resource, action, payload hash, UTC time. Append-only; avoid personal data in logs.

Prototype storage is intentionally one table: analyses(id PRIMARY KEY, fingerprint UNIQUE, created_at, payload). The normalized schema is a production specification, not a deployed database.

End-to-end flow



Implemented API and validation

Endpoint	Input / output and errors
POST /api/analyses	Multipart invoices + bank_statement -> Analysis. 413 size, 422 unreadable/schema, 200 with row blockers for inspectable partial results. Duplicate hashes/version reuse ID.
POST /api/demo/{clean,issues}	Runs included fixtures through the same pipeline. 404 unknown profile.
GET /api/analyses/{id}	Saved Analysis; 404 unknown ID. Add /export for JSON attachment.
GET /api/samples/{name}	Whitelisted fixture; 404 otherwise. GET /api/health returns local status.
POST /api/bank-lab	1-20 sequential status strings or objects with status + timezone-aware observed_at. Returns UNVERIFIED/UNKNOWN, invalid transitions and stale observations.
POST /api/bank-lab/simulate	Seeded independent vs perfectly correlated status-error experiment; illustrative only. No real bank connection.

Invoice columns: invoice_id, customer_id, issue_date, due_date, net_amount, vat_amount, currency. ISO dates, SAR, positive net, non-negative synthetic VAT. Limits: 5 MB/file, 12 MB/request, 10,000 invoice rows, 30 columns, 30 MB expanded XLSX, 40 PDF pages. Unsupported bank layouts are rejected. Local guards are not a complete hostile-upload defense.

Analysis contains id, created_at, version, status, period, sources, counts, metrics, monthly, invoices, customer_risk, aging, anomalies, issues, unmatched, summary, model and assumptions. Each issue has severity, code, source and reason. /openapi.json provides exact typed request contracts; /docs provides Swagger UI. README documents running and response interpretation.

Production handoff contract

POST /v1/requests creates an org-scoped draft. POST /v1/requests/{id}/documents issues scoped uploads; POST /v1/requests/{id}/analyses returns 202/job_id. GET /v1/jobs/{id} exposes progress/failure. POST /v1/packages/{id}/reviews requires version, named approval and consent; reject changed versions with 409. POST /v1/packages/{id}/bank-submissions requires an approved immutable version and Idempotency-Key, returning 202. Persist submission plus outbox atomically. Signed bank events are deduplicated by event ID; GET submission separates observed from verified status. Use 401/403 access, 409 transitions, 422 evidence, 429 limits and 503 transient dependencies. These are design-only endpoints.

Package lifecycle: processing -> blocked or ready_for_review -> approved -> queued -> submitted_unverified -> confirmed. Only a reviewer may approve; changed evidence creates a new version. No last-response-wins updates of confirmed bank state.

Metrics, AI and limitations

Money is integer halalas with ROUND_HALF_UP. Only native float noise within SAR 0.0000001 of a cent is normalized; text / meaningful fractional-cent inputs are rejected. Net sales exclude VAT; collections and receivables include it. Outstanding = gross invoices minus matched receipts. Overdue uses due date strictly before cutoff; 90+ means more than 90 days. Not-overdue includes due today and is not financing eligibility.

Customer payment duration is gross-amount-weighted days from issue to final receipt, **fully paid invoices only**, with sample counts shown. It excludes unpaid invoices and is not DSO. Collection ratio uses matched receipts / gross invoices in the window, so recent not-due invoices affect it. Quarter change is last vs first three months, not year-on-year growth. Costs and debt schedules are absent; profitability and affordability cannot be inferred.

Isolation Forest: 200 trees, seed 42, contamination 0.03; features are log gross amount, terms and log customer-relative amount. Fit per upload, no credit labels. The 3% tail setting is an explicit review-budget heuristic, unrelated to the bank error rate. Scores are not probabilities or measured accuracy. Display the top three flags by outstanding then gross amount; retain all flags in exceptions/export. Model card: docs/model-card.md.

Exact invoice references are the largest matching simplification. Real bank data needs candidate matching by amount/date/customer, ambiguity handling and human confirmation. The synthetic 15% VAT and 35% concentration / 10% overdue triggers are illustrative assumptions, chosen independently of generated outcomes; they are not bank policy.

Task 4 - Integration judgment

Safe plan with a faulty API and no sandbox

1. **Clarify truth and the failure.** Ask what 3% measures, the denominator and time window, which statuses are affected, whether errors cluster, and whether the issue is stale data or incorrect mapping. Agree an authoritative verification channel, transitions, timestamps, reference IDs, escalation owner and response SLA.

2. **Test offline first.** Use agreed schemas / sanitized examples and a fake adapter for wrong/stale statuses, timeouts, duplicate/out-of-order callbacks and rate limits. Include both independent errors and correlated bursts. These tests validate our handling, not the bank's reliability.

3. **Pilot with explicit bank authorization.** Ask for nominated references and a controlled production test, starting read-only with consented data. Never create a fictitious production financing request or unintended financial action. Without a safe verification route, retain a manual handoff.

4. **Submit one logical request.** Commit the immutable approved package and durable outbox atomically. Use a bank-supported idempotency key. Retry transient failures with bounded backoff/jitter and Retry-After. After an ambiguous POST timeout, look up the existing key/reference before retrying; if lookup/deduplication is unavailable, stop for manual confirmation.

5. **Separate observation from confirmation.** Append raw status, reference, source and retrieval timestamps. Valid PENDING -> APPROVED progression is not a conflict; terminal reversals or stale events need investigation. Repeated identical responses cannot independently verify truth. A webhook signature establishes origin, not correctness.

6. **Guard terminal actions and reconcile.** Require authoritative confirmation and named reviewer approval before initial customer-facing terminal actions. Reconcile pilot terminal states at least daily. Pause automation on any confirmed false terminal status or reference mismatch; use a kill switch and manual path. Recovery replays the outbox without duplicate submissions.

Measure verified mismatches with denominator/confidence intervals, false terminal statuses, stale age, ambiguous submissions, duplicate suppression and resolution time. Pilot gates: no duplicate logical submissions, full audit coverage and all terminal actions independently confirmed. Agree sample size/tolerances with the bank; a tiny pilot cannot prove a zero error rate.

Three wrong responses would have probability 0.03^3 only under independent errors conditioned on a known true state. Perfectly correlated errors can remain wrong across all three polls at 3%. Neither is a posterior estimate that an observed approval is correct. The lab demonstrates both assumptions and keeps repeated APPROVED observations unverified; it also handles normal progression, stale timestamps, reversals and timeout outcomes.

Bank call: three concrete requests

“Thanks for flagging the issue. Before we automate status-dependent actions, could we agree on three things?”

- **Meaning and verification:** “What does the 3% represent, and which authoritative source should we use to confirm the actual status? Who owns discrepancies?”
- **Safe retry:** “Can you support a stable idempotency key or lookup by request reference after a timeout, so we never submit the same request twice?”
- **Controlled pilot:** “Without a sandbox, can we authorize a small production test using nominated references, starting read-only with no unintended financial actions, and reconcile results together before expanding?”

“If verification is unavailable, we will retain manual confirmation while the API issue is resolved.”

Task 5 - Automate the request (bonus)

Implemented mock-source automation

The working bonus pulls invoices and a statement over HTTP from two fictional APIs, with scoped demo consent/expiry, recurring scheduling, hash-based no-change detection and persisted run history. Changed snapshots run the same evidence pipeline; invalid rows require human correction. Outages/checksum failures retain the last checkpoint and back off. Revocation stops new reads/publication; already downloaded processing may finish locally. Source automation in the UI demonstrates these states. This is full-snapshot synchronization while a single-worker server runs, not a real ERP/bank connector or production OAuth. Contract/demo: docs/automation.md.

Production improvement and sources

Automate a reusable evidence workspace first: remember entity/account mappings, import new closed months and show a missing-period checklist. This removes repeated preparation while preserving customer review and manual upload fallback.

With consent and provider agreements, use accounting/ERP APIs for invoices, credit notes and allocations; use an approved bank feed/export for transactions and coverage. These are candidate integrations, not claims that any specific bank exposes an API. Never collect banking passwords or scrape authenticated sessions.

Production workflow and human checkpoints

Backfill 12 months, then sync incrementally with provider IDs, cursors, timestamps and consent expiry. Version changed records; do not overwrite history. Validate coverage, bank balances and allocations. Revoked consent stops imports. A reviewer resolves ambiguous batch payments, OCR errors, entity mismatches and gaps; the owner confirms scope/consent and a named reviewer approves one immutable package before submission.

Pilot measures

Customer / bank outcome	Measurement and illustrative target
Less preparation effort	Compare active minutes and p90 to manual baseline; target 50% lower median.
Better completeness	Complete packages / started requests; target 90% without re-upload.
Less reviewer effort	Review minutes and exceptions per 100 receipts; target 30% lower median without degraded discrepancy detection.
Useful bank evidence	First-pass completeness and evidence-related rework vs comparable manual cases; approval rate is not a quality proxy.
Safe operations	Track false-ready packages, incorrect allocation, unauthorized access and duplicates; unresolved critical errors stop rollout.

Targets are proposals, not measured results. Compare similar request complexity, retain fallback, collect customer/bank feedback and expand only when effort improves without reduced evidence quality.

AI disclosure

I used ChatGPT/Codex to generate and refine the implementation, synthetic-data generator, written answers and tests. The application uses local scikit-learn Isolation Forest for exploratory invoice review; the narrative brief is a deterministic template, not an LLM response. All data is fictional, and no customer records or credentials are sent to an external model. Automated execution results and environment details are recorded in docs/validation.md; they do not constitute a real-bank or production validation.

Reproduction and sample integrity

README provides setup and the demo walkthrough; scripts/generate_data.py recreates the seeded scenario. Monthly counts vary, amounts follow customer-specific lognormal distributions and payment behavior differs by customer. INV-202605-010 is a deliberately planted large, partially paid order; retrieval is a sanity check, not model accuracy evidence. Clean means structurally valid, not creditworthy. The dirty fixture introduces a duplicate and three invalid rows, creating blockers. Full limitations, model settings and validation evidence are included with the code.